LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Muhammad Rado Saputra (2609106099)

Kelas (C2 ‘26)

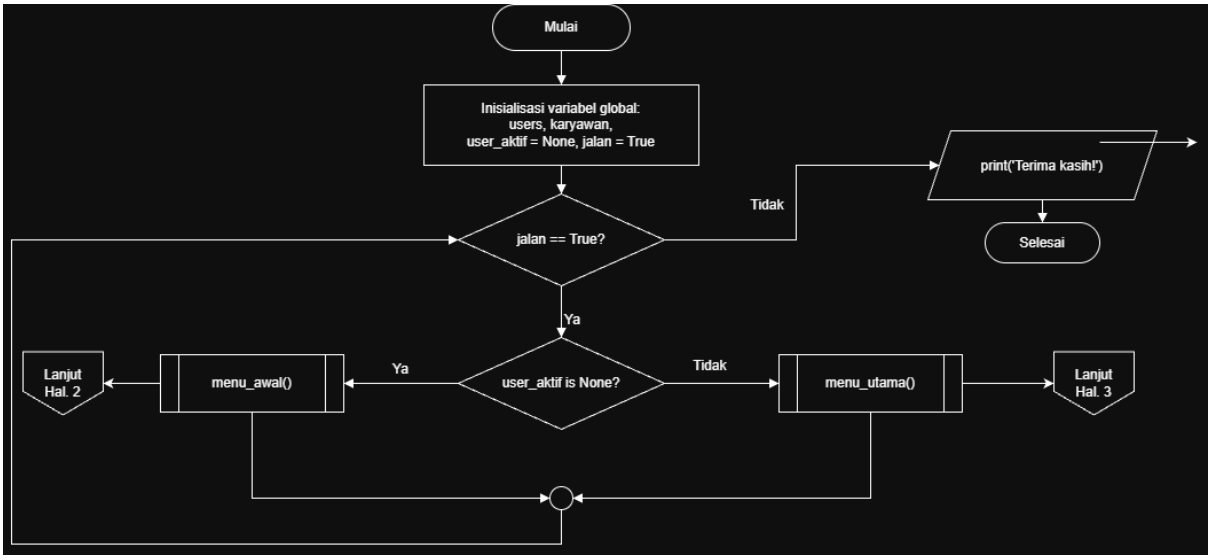
PROGRAM STUDI INFORMATIKA

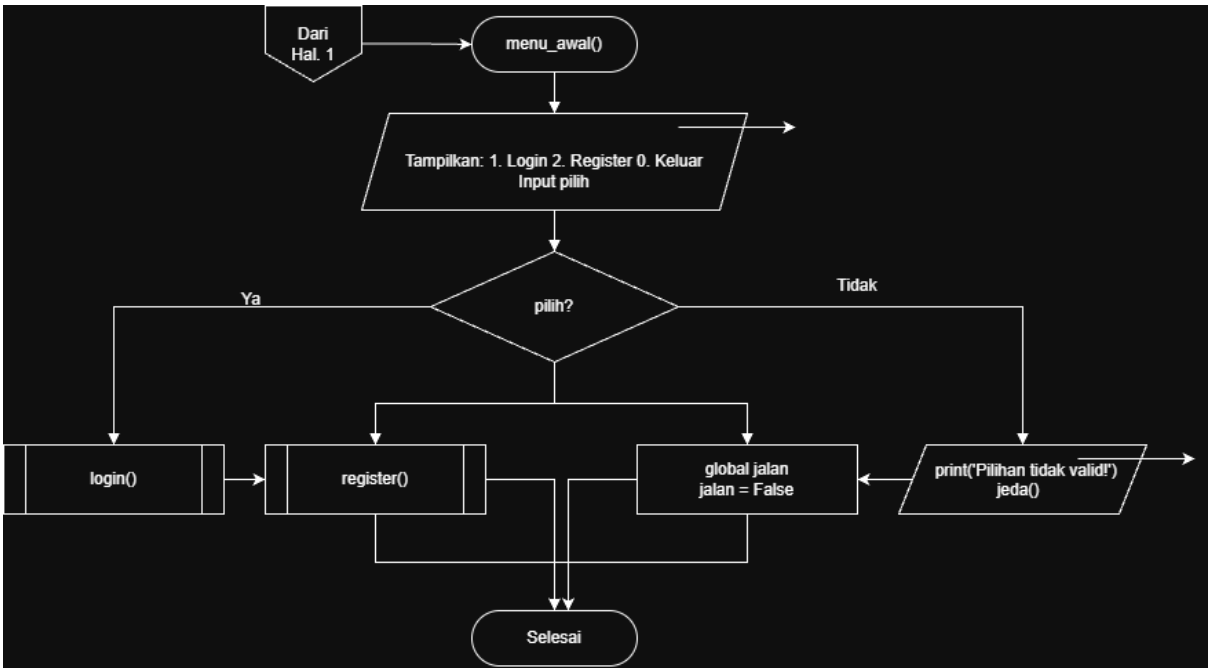
UNIVERSITAS MULAWARMAN

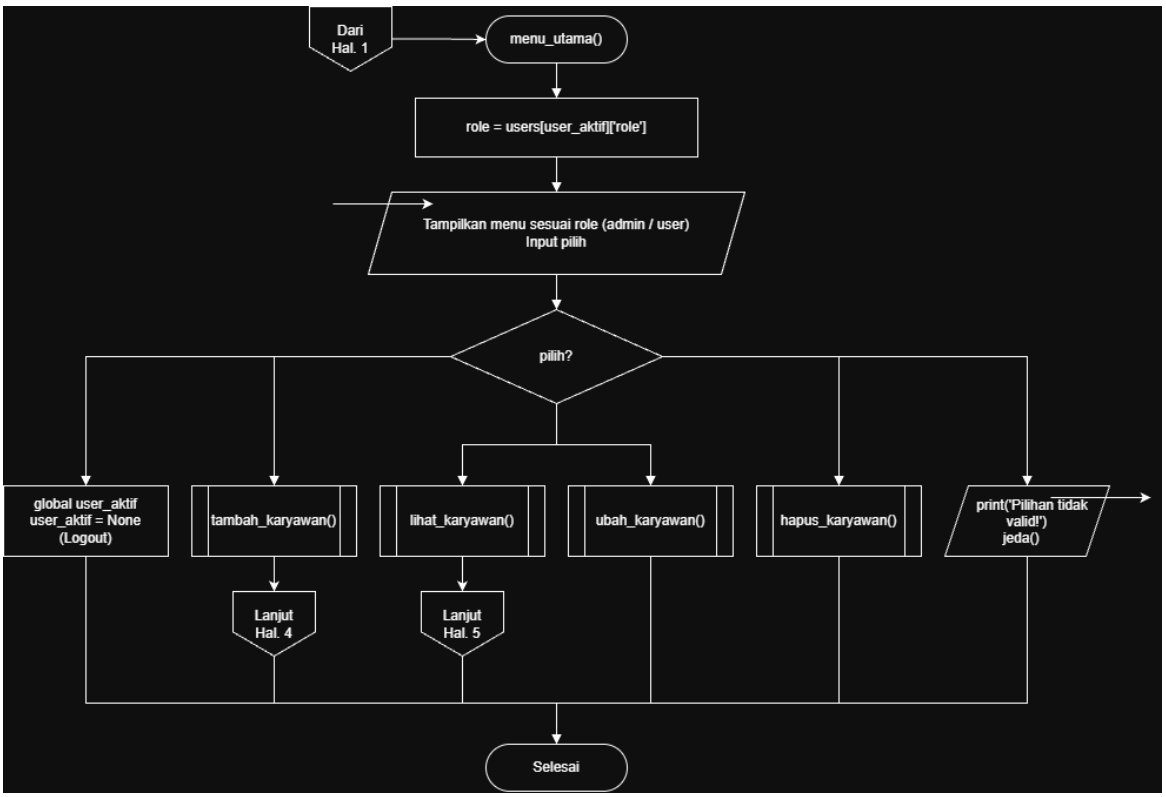
SAMARINDA

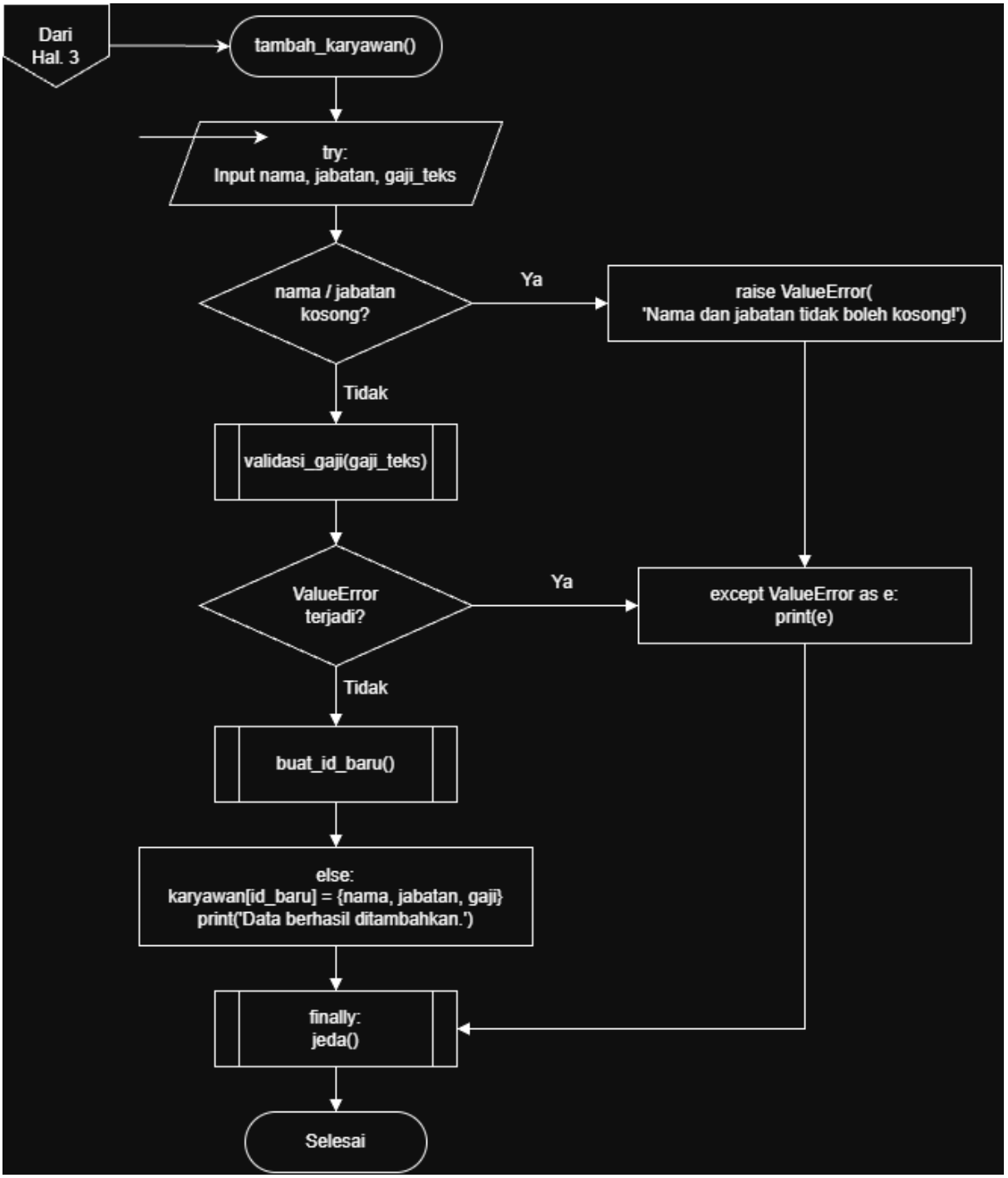
2026

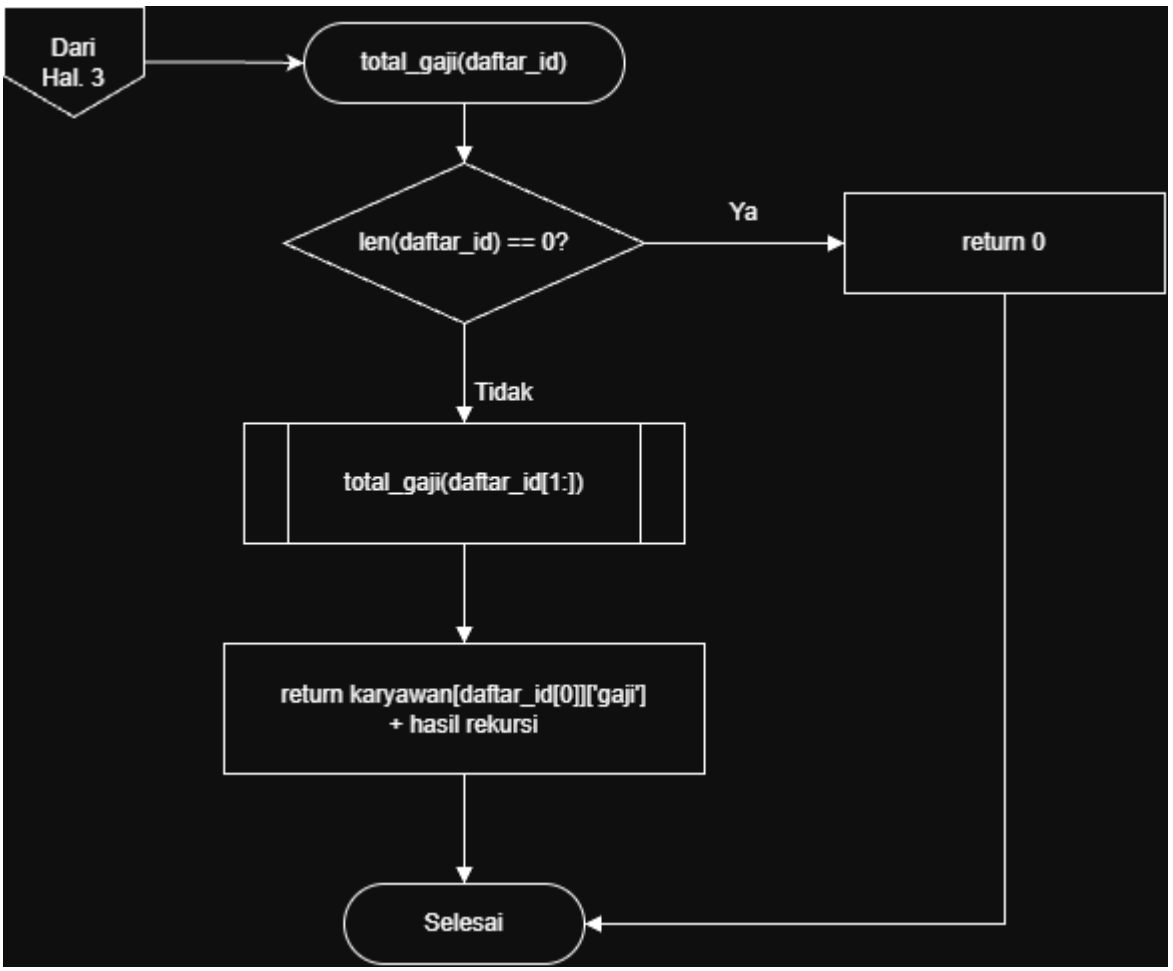
1. Flowchart


Mulai
Inisialisasi variabel global:
users, karyawan,
user_aktif = None, jalan = True
jalan == True?
Tidak
print('Terima kasih!')
Selesai
Ya
user_aktif is None?
Ya
menu_awal()
Lanjut
Hal. 2
Tidak
menu_utama()
Lanjut
Hal. 3


Dari
Hal. 1
menu_awal()
Tampilkan: 1. Login 2. Register 0. Keluar
Input pilih
pilih?
Ya
Tidak
login()
register()
global jalan
jalan = False
print('Pilihan tidak valid!')
jeda()
Selesai


Dari
Hal. 1
menu_utama()
role = users[user_aktif]['role']
Tampilkan menu sesuai role (admin / user)
Input pilih
pilih?
global user_aktif
user_aktif = None
(Logout)
tambah_karyawan()
lihat_karyawan()
ubah_karyawan()
hapus_karyawan()
print('Pilihan tidak
valid!')
jeda()
Lanjut
Hal. 4
Lanjut
Hal. 5
Selesai


Dari
Hal. 3
tambah_karyawan()
try:
Input nama, jabatan, gaji_teks
nama / jabatan
kosong?
Ya
raise ValueError(
'Nama dan jabatan tidak boleh kosong!')
Tidak
validasi_gaji(gaji_teks)
ValueError
terjadi?
Ya
except ValueError as e:
print(e)
Tidak
buat_id_baru()
else:
karyawan[id_baru] = {nama, jabatan, gaji}
print('Data berhasil ditambahkan.')
finally:
jeda()
Selesai



Flowchart ini menjelaskan saat program dijalankan, tampil menu awal Login, Register, atau Keluar. Kalau memilih register, kita memasukkan username dan password baru. Program menolak kalau kosong atau username sudah dipakai, dan kalau berhasil akun akan disimpan sebagai user biasa. Kalau memilih Login, program mengecek username dan password ke daftar akun, dan kalau tidak cocok tampil pesan salah lalu kembali ke menu awal. Setelah login, menu menyesuaikan role. Admin bisa Tambah, Lihat, Ubah, dan Hapus karyawan. User biasa hanya bisa Lihat. Tambah meminta nama, jabatan, dan gaji (gaji harus angka lebih dari 0), lalu ID dibuat otomatis. Lihat menampilkan semua karyawan. Ubah dan Hapus meminta ID karyawan, dan kalau ID tidak ada tampil "ID tidak ditemukan!". Di Ubah, kolom yang dikosongkan tidak berubah. Pilih 0 untuk logout dan kembali ke menu awal, lalu pilih 0 lagi untuk menutup program.

2. Deskripsi Singkat Program

Tujuan program ini adalah membantu mengelola data karyawan seperti ID, nama, jabatan, dan gaji dengan lebih rapi, sekaligus membatasi siapa yang boleh mengubahnya. Fungsinya, pengguna bisa daftar dan masuk dengan akunnya sendiri. Admin bisa menambah, melihat, mengubah, dan menghapus data karyawan, sedangkan pengguna biasa hanya bisa melihatnya. Kalau ada input yang keliru, seperti kolom kosong, gaji bukan angka, atau ID tidak ditemukan, program langsung memberi tahu supaya data tetap benar.

3. Source Code

A. Input

Fungsinya menerima data dari pengguna

Source Code:

```
pilih = input("Pilih: ").strip().
```

B. Print

Fungsinya menampilkan pesan dan hasil ke layar

Source Code:

```
print("1. Login\n2. Register\n0. Keluar")
```

C. Import os

Memanggil modul os supaya program bisa memakai fitur sistem operasi, salah satunya untuk membersihkan layar

Source Code:

```
import os
```

D. Os system

Menjalankan perintah terminal dari dalam Python.

Source Code:

```
os.system("cls" if os.name == "nt" else "clear")
```

E. Dictionarry

List yang isinya list lagi, dipakai untuk menyimpan data

Source Code:

```
users = { "admin": {"password": "admin123", "role": "admin"}, "rado":
{"password": "rado123", "role": "user"}, }

karyawan = { 1: {"nama": "Andi Pratama", "jabatan": "Manager", "gaji":
12000000}, 2: {"nama": "Ahmad Rizki", "jabatan": "Staff", "gaji": 6000000}, 3:
{"nama": "Rudi", "jabatan": "Programmer", "gaji": 8000000}, }
```

F. Strip

Membuang spasi di awal dan akhir teks, supaya input yang tidak sengaja diberi spasi tidak dianggap salah.

Source Code:

```
username = input("Username: ").strip()
```

G. While

Perulangan yang jalan terus selama kondisinya benar.

Source Code:

```
while user_aktif is not None:
```

H. append

Menambahkan data baru di akhir list.

Source Code:

```
users.append([username, password, "user"])
```

I. For dan items()

Perulangan yang menelusuri isi dictionary satu per satu. Method items() mengambil key (id_k) dan value (k) sekaligus.

Source Code:

```
for id_k, k in karyawan.items():
    print("ID :", id_k)
    print("Nama :", k["nama"])
```

J. F-string

Berfungsi untuk menyisipkan variabel atau ekspresi matematika langsung ke dalam teks secara cepat dan rap

Source Code:

```
print(f"=== MENU ({user_aktif[0]} - {role}) ===")
```

K. Del

Menghapus sebuah key beserta isinya dari dictionary.

Source Code:

```
del karyawan[int(id_hapus)]
```

L. Isdecimal

Mengecek apakah teks hanya berisi angka 0-9, sebelum diubah dengan int(). Dengan begitu input huruf tidak membuat program error.

Source Code:

```
if not id_hapus.isdecimal() or int(id_hapus) not in karyawan:
    print("ID tidak ditemukan!")
```

M. Variable global

Variabel yang dibuat di luar fungsi sehingga dapat diakses oleh semua fungsi dan prosedur

Source Code:

```
users = {
    "admin": {"password": "admin123", "role": "admin"},
    "rado": {"password": "rado123", "role": "user"},
}

karyawan = {
    1: {"nama": "Andi Pratama", "jabatan": "Manager", "gaji": 12000000},
    2: {"nama": "Ahmad Rizki", "jabatan": "Staff", "gaji": 6000000},
    3: {"nama": "Rudi", "jabatan": "Programmer", "gaji": 8000000},
}

user_aktif = None
jalan = True
```

N. Variable Lokal

Variabel yang dibuat di dalam fungsi sehingga hanya dapat diakses di dalam fungsi tersebut

Source Code:

```
def tambah_karyawan():
    try:
        nama = input("Nama    : ").strip()
        jabatan = input("Jabatan : ").strip()
        gaji_teks = input("Gaji    : ").strip()
        if nama == "" or jabatan == "":
            raise ValueError("Nama dan jabatan tidak boleh kosong!")
        gaji = validasi_gaji(gaji_teks)

jalan = True
```

O. Del

Variabel yang dibuat di dalam fungsi sehingga hanya dapat diakses di dalam fungsi tersebut

Source Code:

```
del karyawan[id_hapus]
```

P. Try, Except, Else, Finally

Mekanisme error handling agar program tidak berhenti saat terjadi kesalahan.

Source Code:

```
Source Code:

def tambah_karyawan():
    try:
        nama = input("Nama    : ").strip()
        jabatan = input("Jabatan : ").strip()
        gaji_teks = input("Gaji    : ").strip()
        if nama == "" or jabatan == "":
            raise ValueError("Nama dan jabatan tidak boleh kosong!")
        gaji = validasi_gaji(gaji_teks)
    except ValueError as e:
        print(e)
    else:
        karyawan[buat_id_baru()] = {"nama": nama, "jabatan": jabatan, "gaji":
gaji}
        print("Data berhasil ditambahkan.")
    finally:
        jeda()
```

Q. Raise

Membuat error secara manual ketika suatu kondisi tidak terpenuhi.

Source Code:

```
# Di dalam baca_id()
if not teks.isdecimal():
    raise ValueError("ID harus berupa angka!")
    id_k = int(teks)
    if id_k not in karyawan:
    raise KeyError(id_k)

# Penanganan di hapus_karyawan()
try:
    tampilkan_daftar_singkat()
    id_hapus = baca_id()
except KeyError:
    print("ID tidak ditemukan!")
except ValueError as e:
    print(e)
```

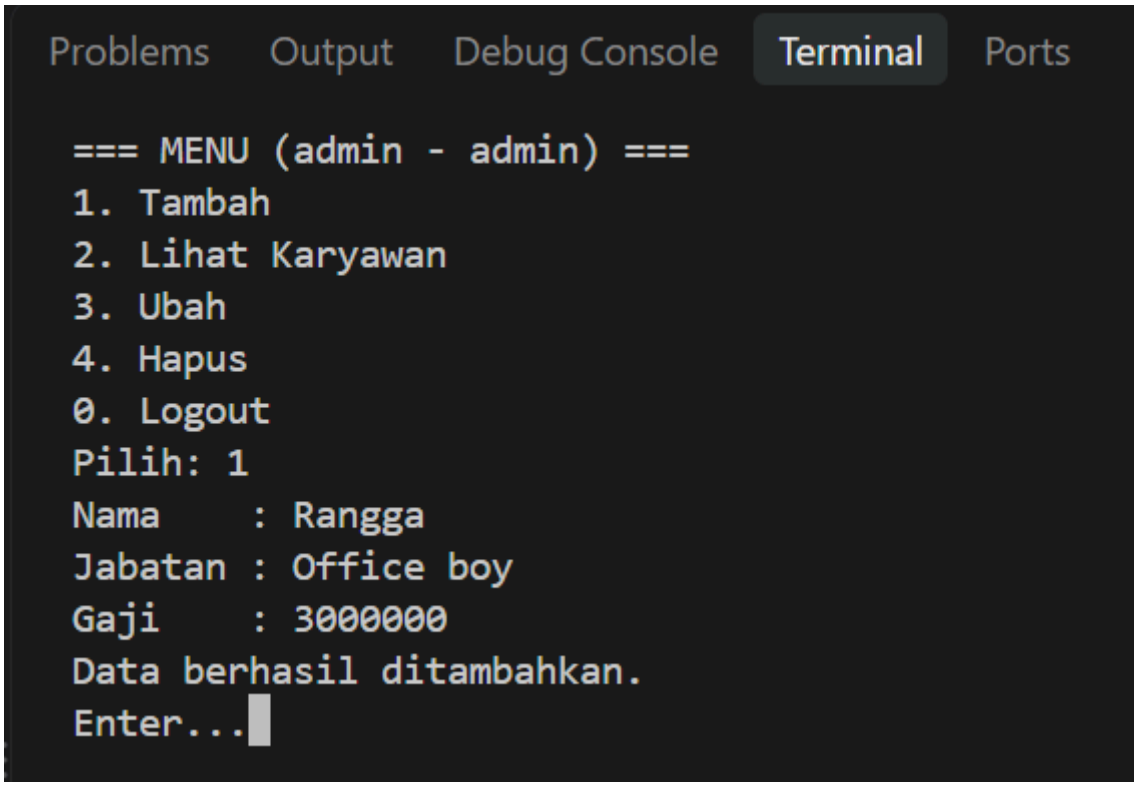
R. Keyword global

Digunakan agar fungsi dapat mengubah nilai variabel global. Tanpa global, Python menganggap user_aktif sebagai variabel lokal baru di dalam fungsi.

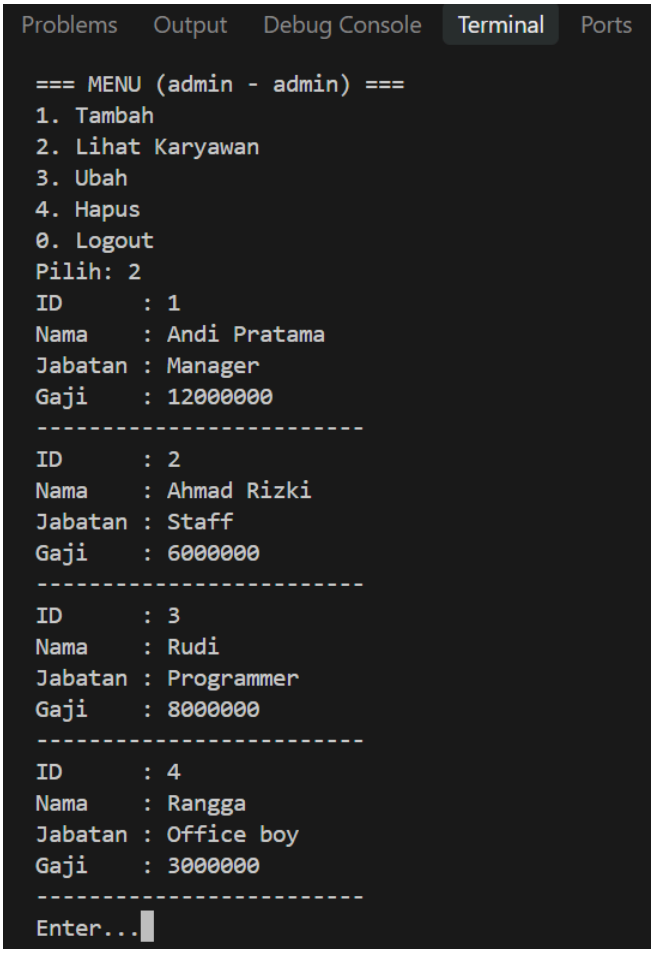
Source Code:

```
def login():
    global user_aktif
    try:
        username = input("Username: ").strip()
        password = input("Password: ").strip()
        if not cek_login(username, password):
            raise ValueError("Username atau password salah!")
    except ValueError as e:
        print(e)
    else:
        user_aktif = username
```

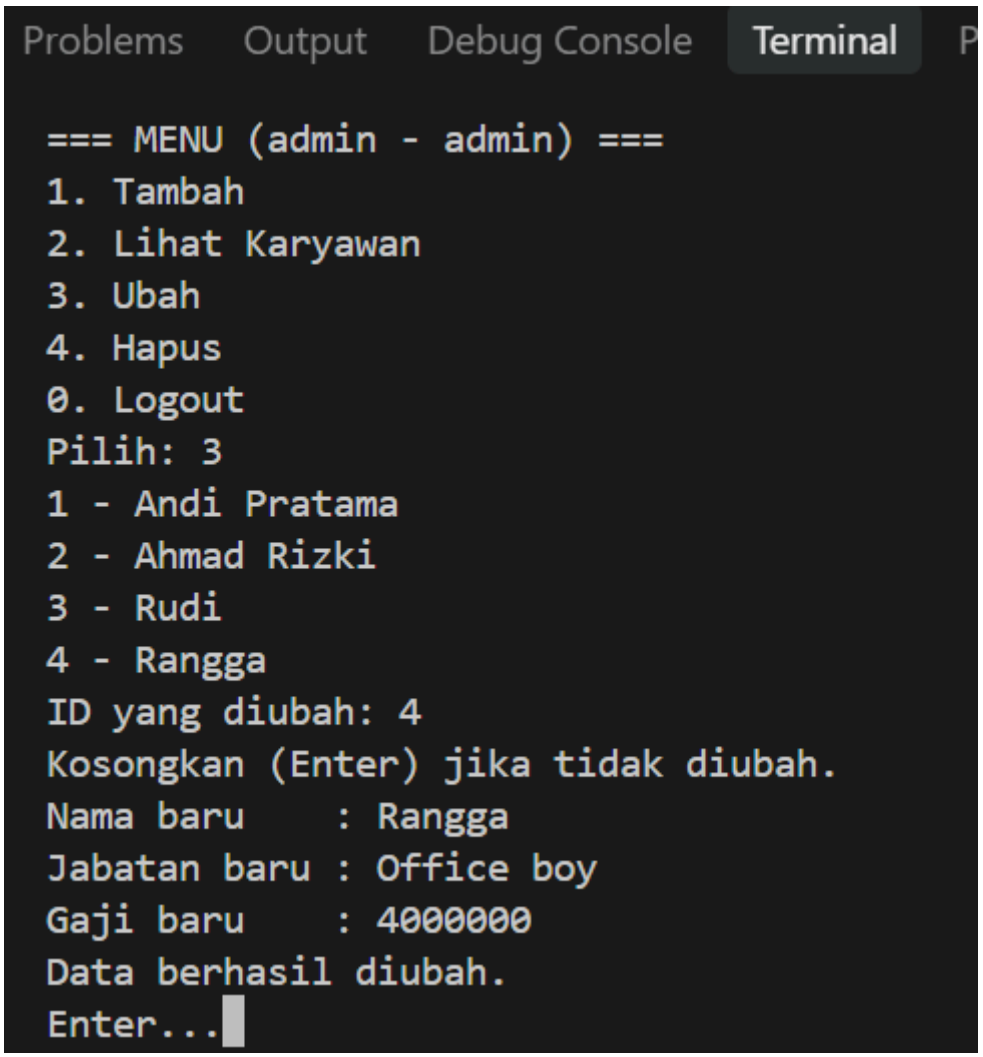
4. Hasil Output



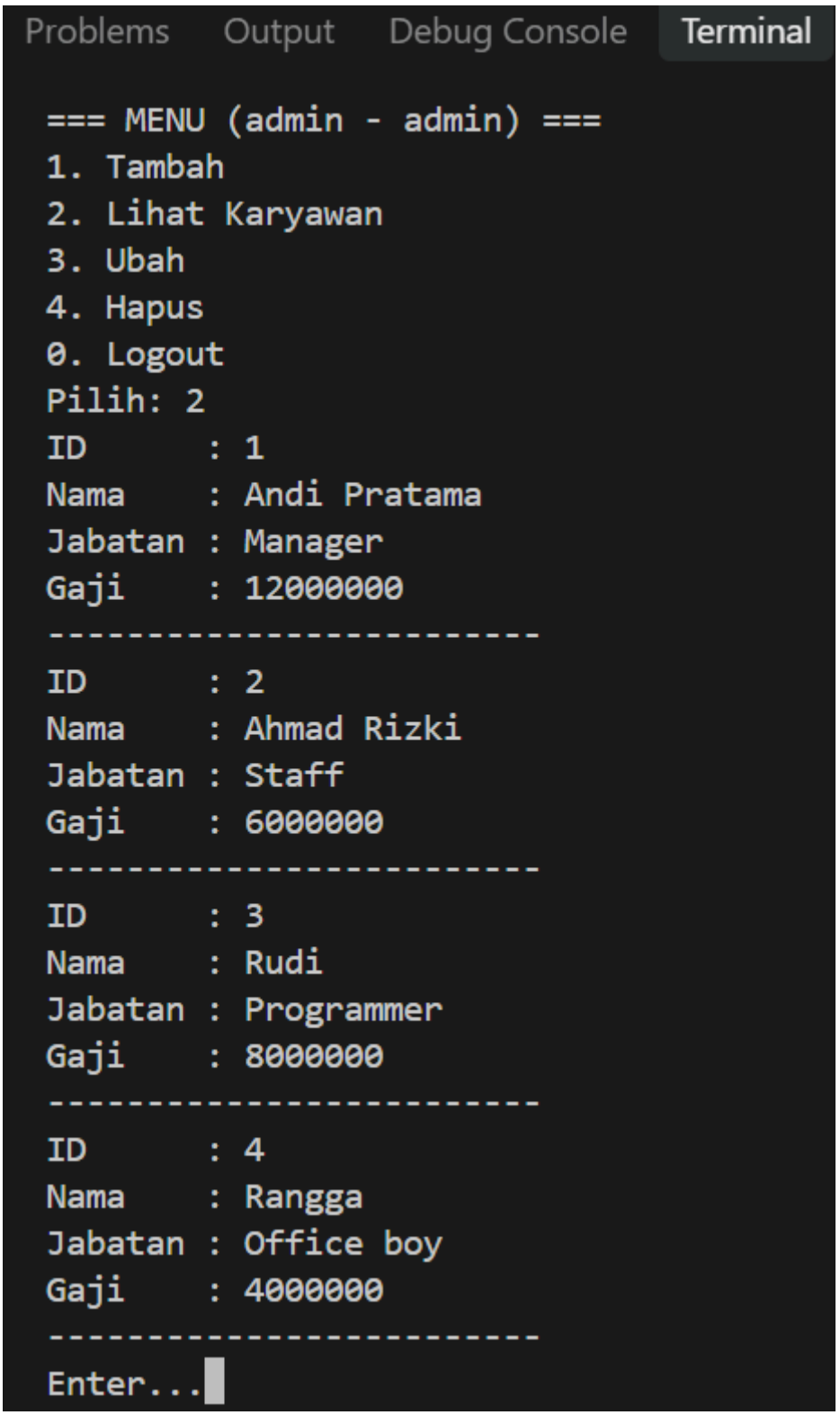
Gambar 4.1 Role admin menambahkan karyawan baru



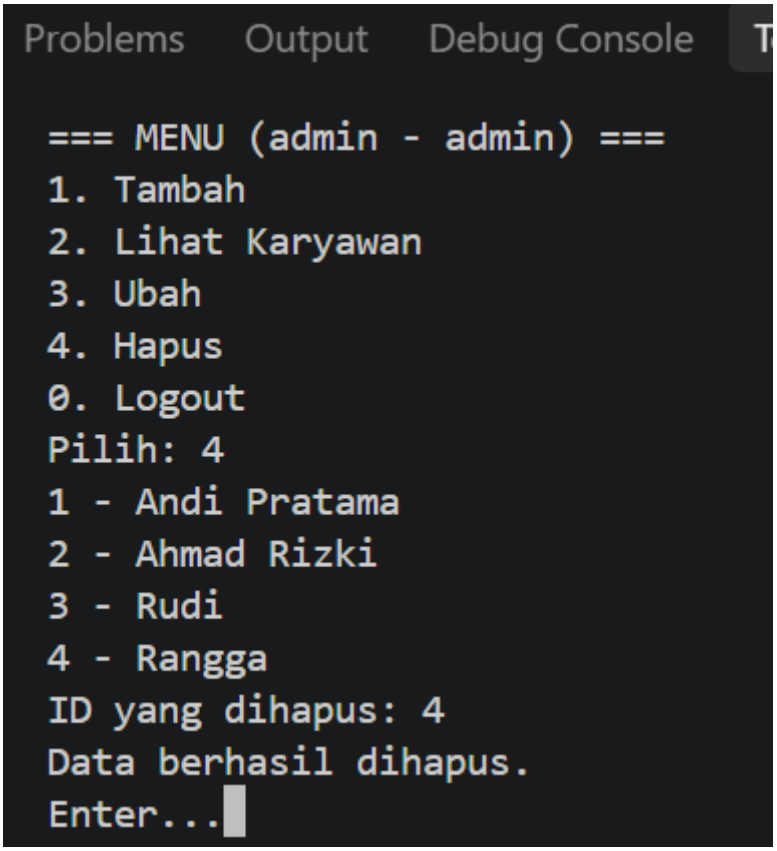
Gambar 4.2 Karyawan yang baru ditambahkan masuk ke dalam list



Gambar 4.3 Mengubah data karyawan



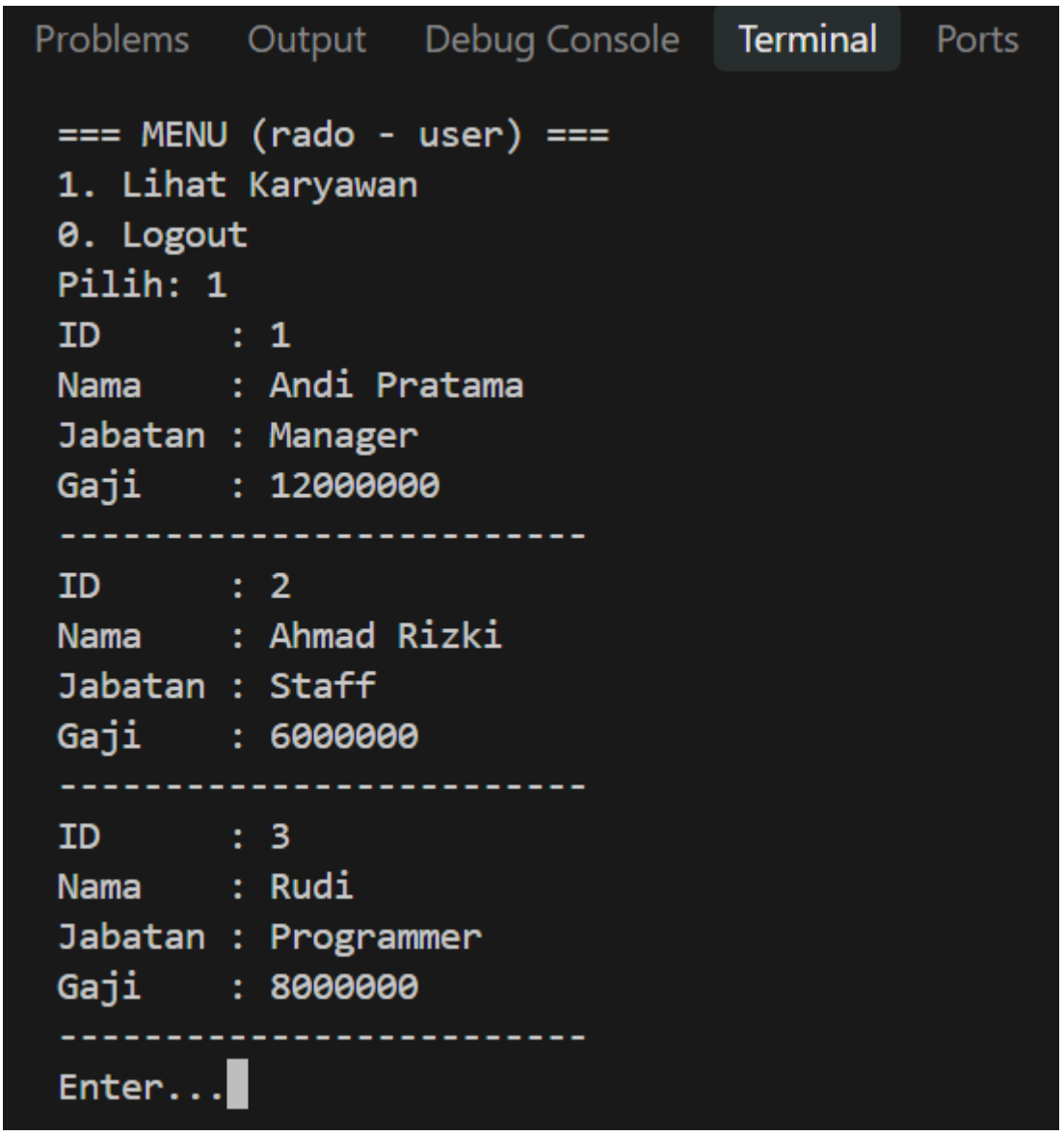
Gambar 4.4 Data berhasil di ubah



Gambar 4.5 Menghapus data karyawan



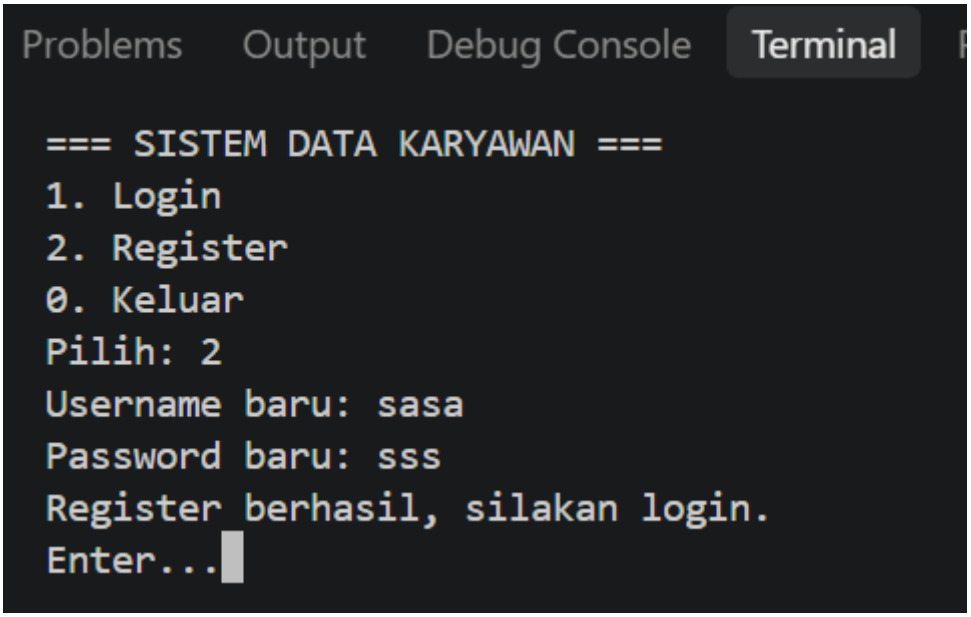
Gambar 4.6 Data karyawan berhasil di hapus



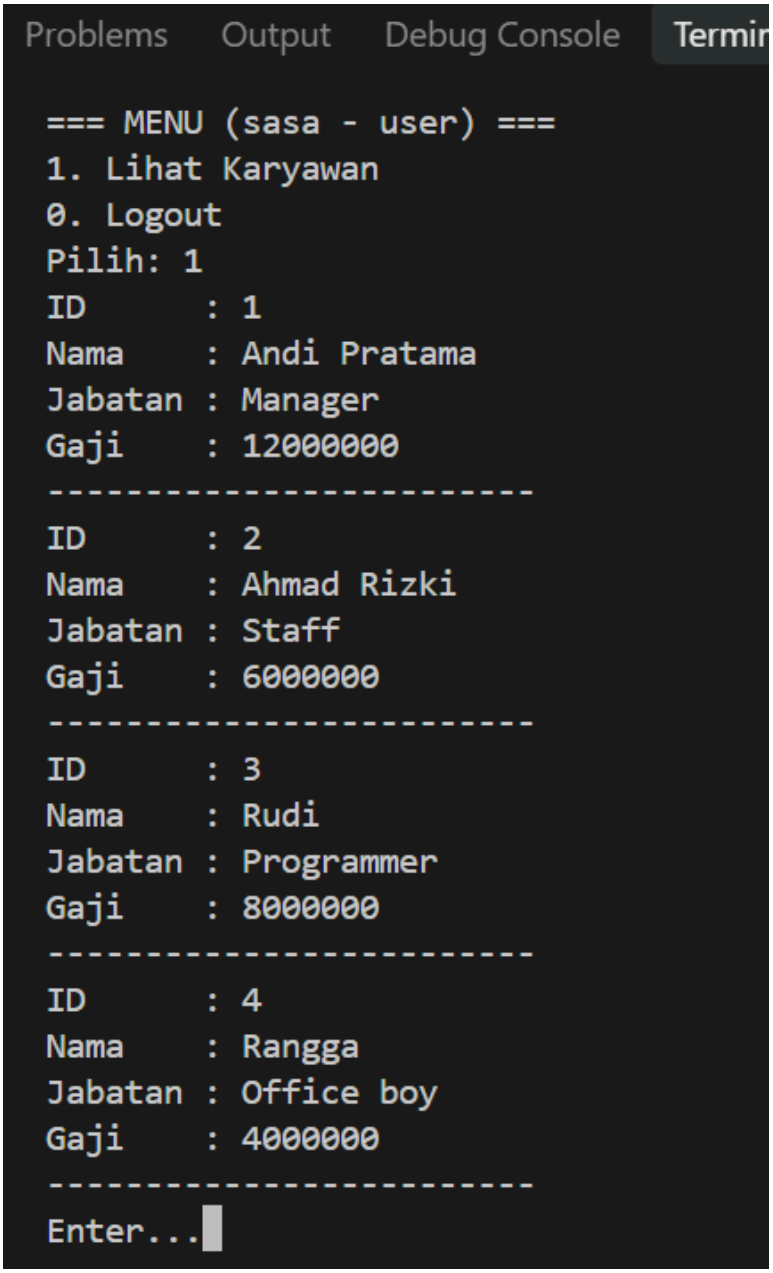
Gambar 4.7 Role user dan melihat karyawan



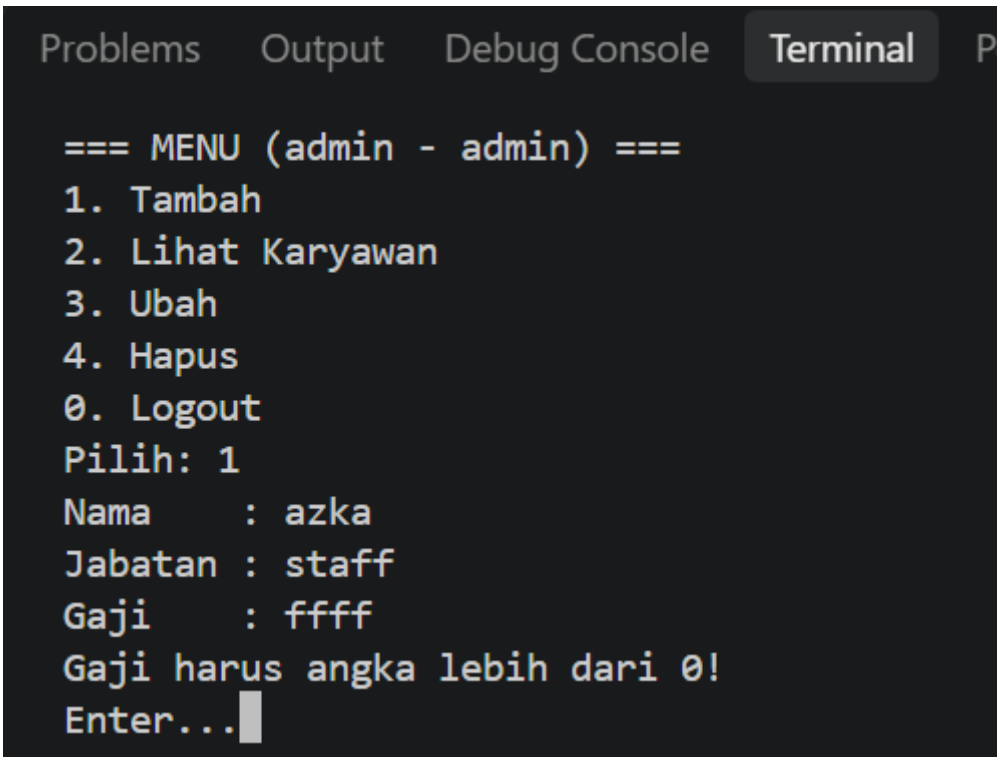
Gambar 4.8 Role user melihat data karyawan yang baru saja di tambahkan



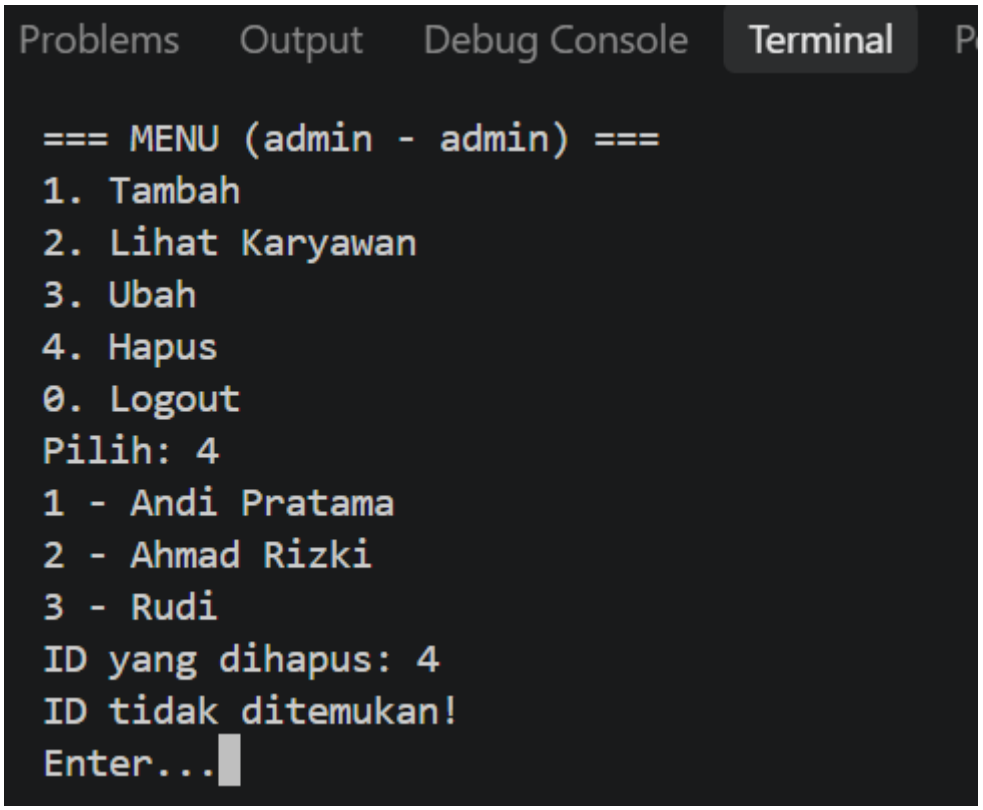
Gambar 4.9 Register role user baru



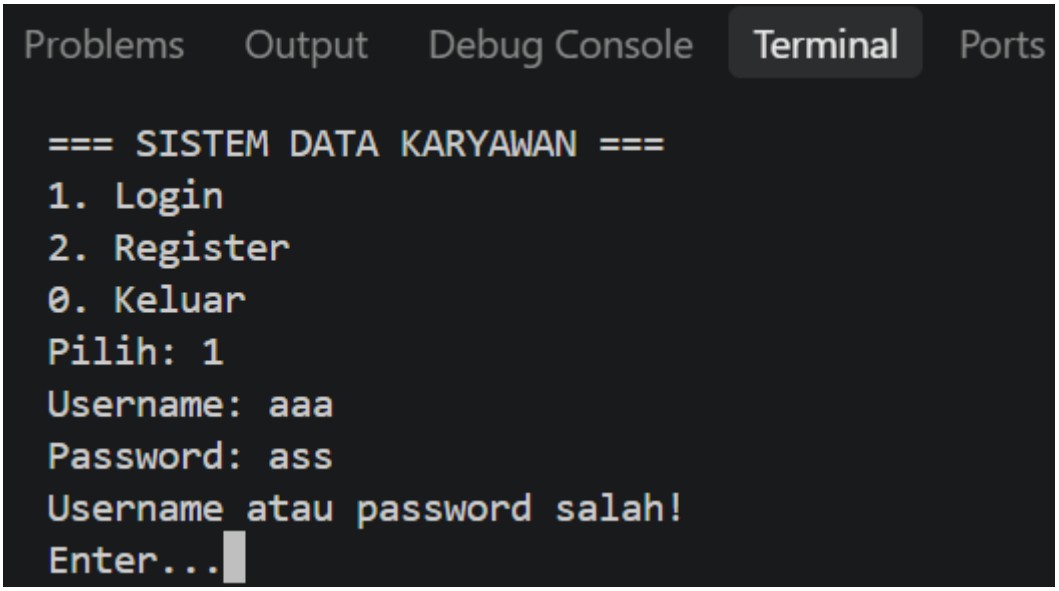
Gambar 4.10 Role user yang baru melihat data karyawan



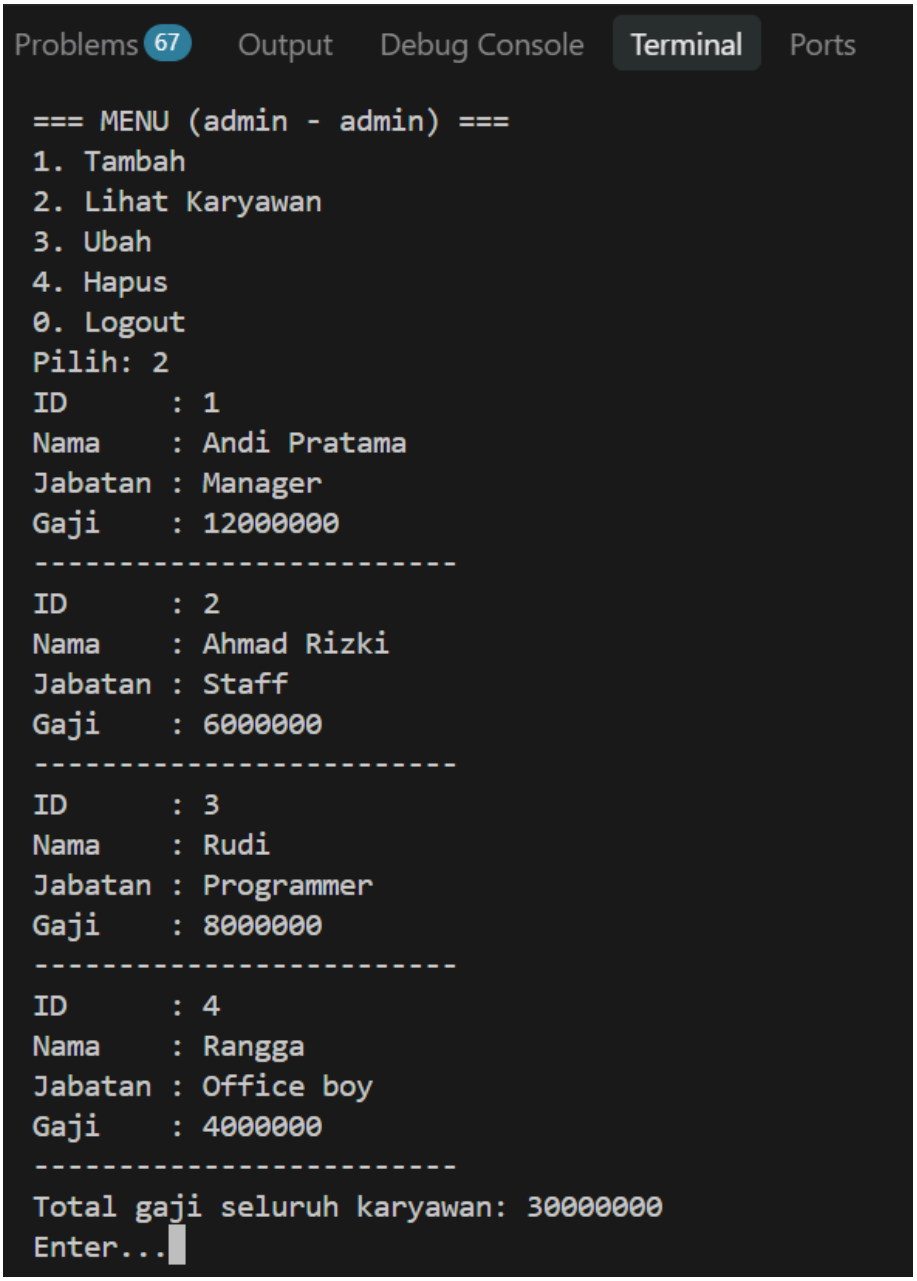
Gambar 4.11 Jika mengisi gaji tidak menggunakan angka



Gambar 4.12 jika menghapus data karyawan yng tidak ada di dalam list



Gambar 4.13 jika memasukkan username atau password yg salah



Gambar 4.14 Penerapan fungsi rekursif pada total gaji seluruh karyawan

5. Git Add

Git Add digunakan untuk memasukkan perubahan file atau kode ke dalam antrean sebelum disimpan ke Git.

```
PS D:\Praktikum-apd-2026\posttest\posttest-apd-7> git add .
```

6. Git Commit

Git Commit digunakan menyimpan secara permanen perubahan kode yang sebelumnya sudah kita daftarkan di Git Add.

```
PS D:\Praktikum-apd-2026\posttest\posttest-apd-7> git commit -m "pt"
```

7. Git Push

Git Push digunakan untuk mengirim semua riwayat perubahan kode yang sudah kita commit ke reposity di GitHub.

```
PS D:\Praktikum-apd-2026\posttest\posttest-apd-7> git push
```